

AI 기반 KCMVP 사전 검증 시스템

규칙 탐지와 LLM 의미론적 분석의 하이브리드 모델

조수빈, 박도윤, 임다은, 김재환, 정수민, 형유림, 서화정

Table of Contents

1 배경

2 시스템 설계

3 성능 평가

4 결론

KCMVP

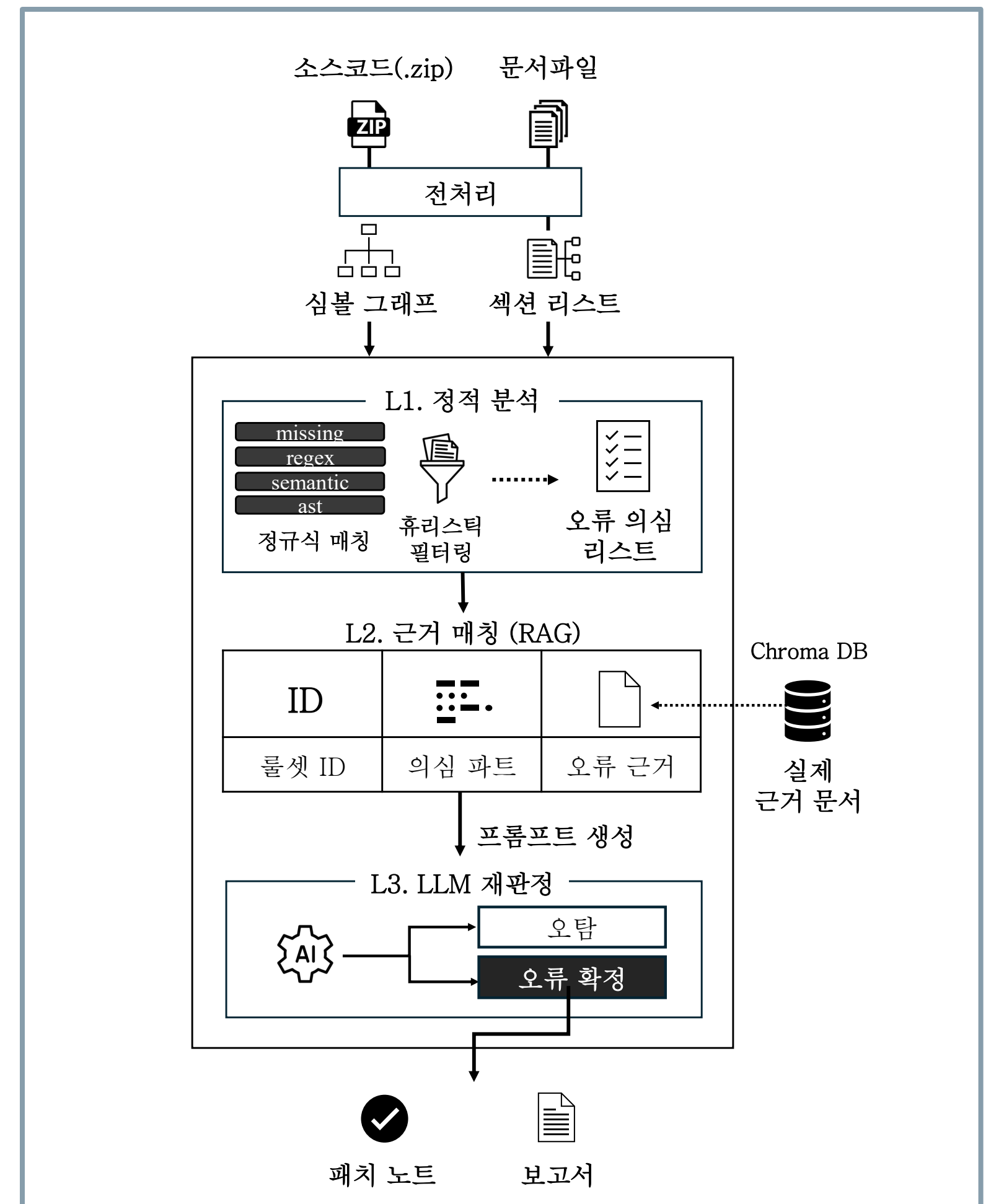
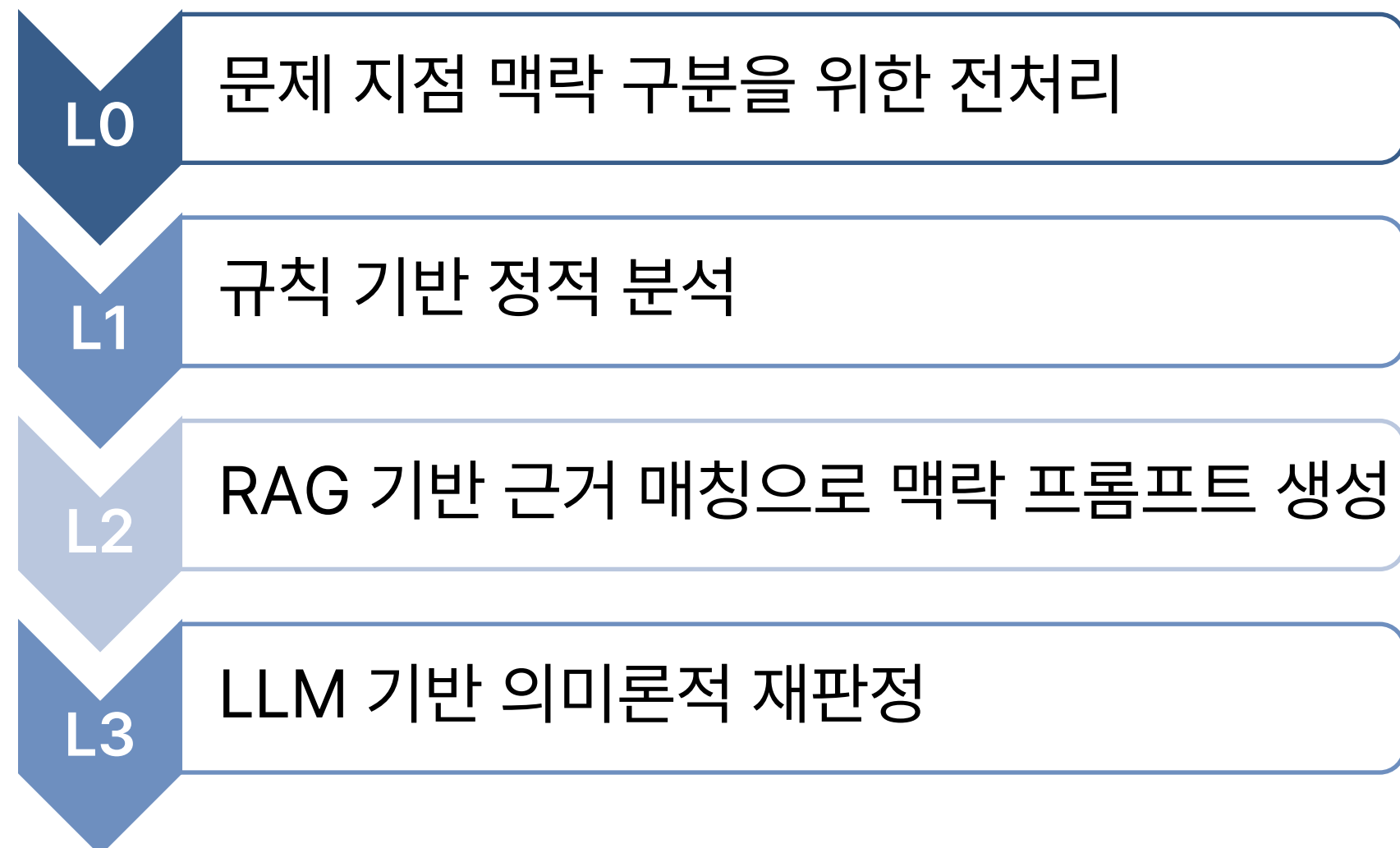
- 한국 암호모듈 검증제도(KCMVP)는 정부 및 공공기관의 정보 시스템에 도입되는 암호모듈에 대한 필수적인 법정 인증 제도
- 검증제도의 전체 프로세스는 통상 1년 반 정도가 소요되며, 결함으로 인한 보완 요청이 발생할 경우 수 주에서 수개월의 추가 지연을 야기
- 반복적 지연은 검증 비용 증가와 제품 출시 지연으로 이어짐

KCMVP 사전 자동 점검 도구에 대한 필요성이 증가

파이프라인 설계

전체 파이프라인은 퍼널형 구조로 설계

- 오탐을 순차적으로 축소하는 방식



L0: 전처리

소스코드와 문서를 이후 분석 단계를 위해 **구조화된 데이터**로 변환

전처리		
소스코드 전처리	소스코드 구조 파악	문서 전처리
• 파일 유형 분류 (benchmark, test 유형 구분) • Thin wrapper 판별 • 주석 제거	• AST 파싱 • 심볼 그래프	• 일반 PDF 구조 추출 • 스캔 PDF OCR 변환

L0: AST, SymbolGraph

L0 코드 전처리는 **AST**와 **Symbol Graph** 두 개의 자료 구조를 생성

AST는 단일 파일 내 코드의 구성 흐름 추약을 산출

- 따라서 파일 경계를 넘지 않으며, 파일 내 흐름을 보는 것을 목적으로 함

함수 내부 제어 흐름	연산 패턴	변수 선언·초기화	함수 호출 위치
-------------	-------	-----------	----------

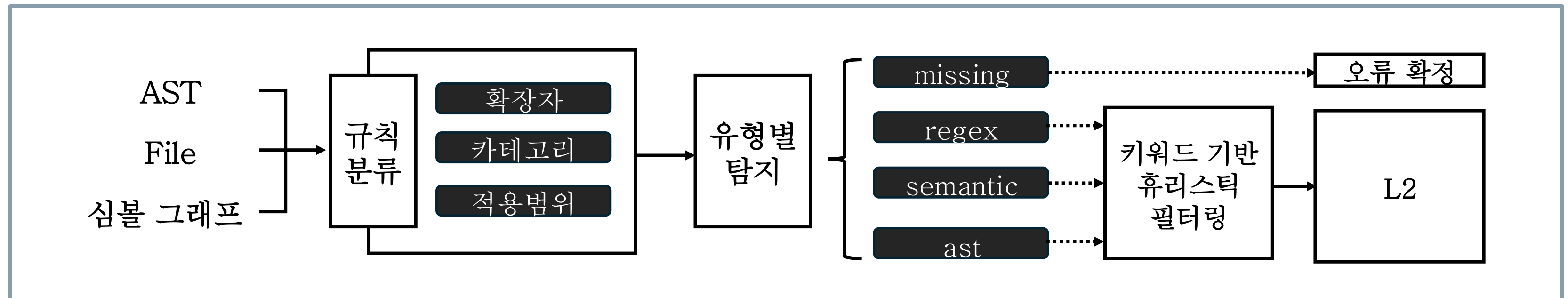
Symbol Graph는 파일 간의 관계 그래프를 산출

- 전체 파일의 구조, 맥락, 호출 관계 파악을 목적으로 함

파일 간 호출 관계	간접 호출 체인	정적 배열 초기화값	전역 타입 역변환
------------	----------	------------	-----------

L1: 규칙 기반 정적 분석

KCMVP 기반의 점검 규칙을 **네 가지 패턴 유형으로 분류**하여 **위반 후보 집합**을 생성



- 입력으로는 **소스파일, 문서원문, 주석**이 주어짐

L1에서는 이를 기반으로 프로젝트 전체에 대해 분석하고, 발견된 위반을 레코드에 누적

L1: 규칙 범주 구분

규칙 범주와 패턴 유형을 구분하여 **점검 대상**과 **책임 범위**를 규정함

- 규칙 범주를 다음과 같이 세분화하여 **주요 점검 항목**을 구분함

	공통 보안		LEA		운영 모드		문서		추적성	
--	-------	--	-----	--	-------	--	----	--	-----	--

- 네가지 패턴 유형에 따라 요구하는 증거의 성격이 다르므로, **후속 처리**를 달리 함

규칙 유형	설명	L3
missing	반드시 구현되어야 할 기능의 누락 (패턴 누락)	X
regex	금지되거나 잘못된 표현 이 포함	선택적
semantic	코드 문맥·의미를 파악 해야만 위반 여부 판단 가능	우선
ast	코드 문법 구조 자체 가 위반인 경우	우선

L1: 휴리스틱 필터링

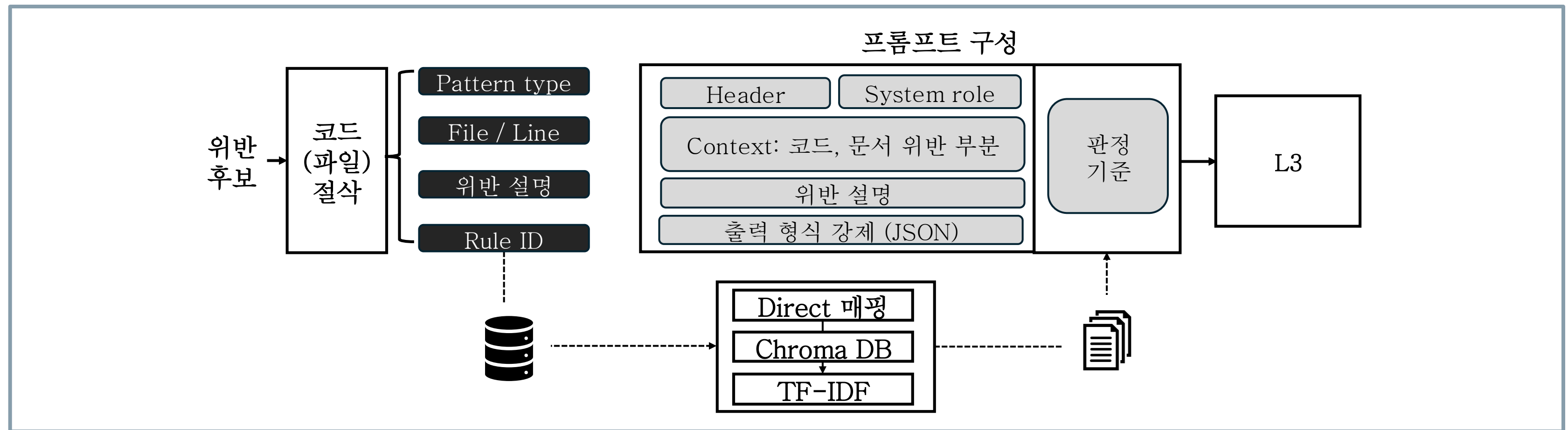
L1이 단순히 패턴을 찾기만 하면 오탐이 매우 많이 발생하게 됨

- 실제 코드 분석 과정 기반에서 주로 발생했던 FP 유형에 대응하여 휴리스틱 필터 적용
- 이를 통해 LLM 비용·시간을 절감하고자 함

휴리스틱 필터	필터링 예시	필터 기준
키워드 필터	하드코딩된 배열의 S-box, 룩업테이블, 테스트 벡터	키워드(sbox, ...)
rand()	타이머 초기화나 배열 인덱스 선택에서 rand()가 사용	주변 코드 문맥
중복 제거	같은 배열 내 위반이 여러 줄에 걸쳐 있을 경우 (중복 위반)	파일 구조
파일 유형	벤치마크나 테스트 용 소스파일에 위반이 포함되는 경우	파일명, 내용 구조

L2: RAG 기반 근거 매칭

L1의 위반 후보에 KCMVP 기반 근거를 부착하여 **L3판정을 위한 맥락**을 구성



- RAG: LLM이 답을 생성할 때 신뢰할 수 있는 데이터베이스에서 정보를 검색하고, 그 내용을 바탕으로 답하여 AI의 허위 정보 생성을 줄이는 기술

L2: 가이드라인 DB 구조

실제 공식문서를 섹션별 **.md 파일**로 구성하여 데이터베이스를 설계

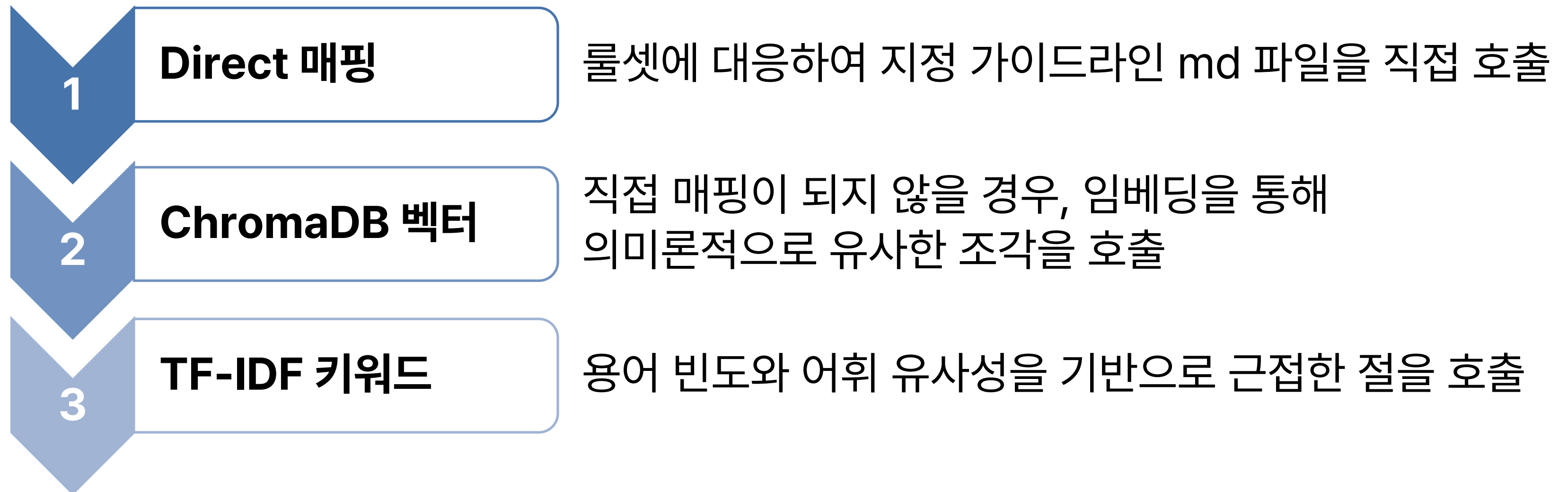
- 이 때 데이터베이스는 KISA의 구현 안내서, KISA LEA 소스코드, 한국표준정보망 요구사항을 기반으로 섹션별로 나누어 구성하였음

섹션	내용
YAML frontmatter	룰셋(ruleID), 심사 항목번호(itemID), 참고 문서, title 등 문서 정보
개요	규칙 개요 (ex. 민감 보안 파라미터는 ... 즉시 제로화해야 한다.)
요구사항	요구사항 (ex. 함수 반환 전, ... 모든 경로에서 제로화가 수행되어야 한다.)
허용 함수 / 금지 함수	코드의 경우 허용 함수(ex. memset_s), 금지 함수 등의 FP 판정 근거
위반 패턴	위반 패턴 (프롬프트의 few-shot 예시)
올바른 구현	올바른 구현 (프롬프트의 few-shot 예시)
참고	실제 참고한 표준 조항의 원문

L2: 3계층 검색 구조

근거 검색은 세 단계로 나누어, 가장 **신뢰할 만한 경로를 우선적으로 시도**

- 원문이 직접 연결 → 의미 기반 검색 → 키워드·통계적 유사도의 **폴백 구조**



L2 존재 유무 비교

L2는 LLM이 항상 동일한 KCMVP 기준을 참조하도록 만드는 **앵커 역할**을 함

항목	L2 부재	L2 존재
L3 판정 문맥	ruleID + 코드	ruleID + 코드 + KCMVP 표준
임계값 오판	LLM 추론에 의존하여 발생 가능	표준 근거로 억제
일관성	대답 때마다 판단이 달라질 가능성	가이드라인 기반 일관성 유지
허용 예외 판단	불안정	예시를 통한 안정

또한 사용자가 위반 탐지에 대한 표준 근거를 확인할 수 있도록 도움

- 보고서에 인용되는 근거와 패치 노트 또한 RAG를 기반으로 작성

프롬프트 최종 생성

프롬프트는 **판단을 위한 배경 지식 삽입, 단계별 추론, 후처리**의 7계층 구조

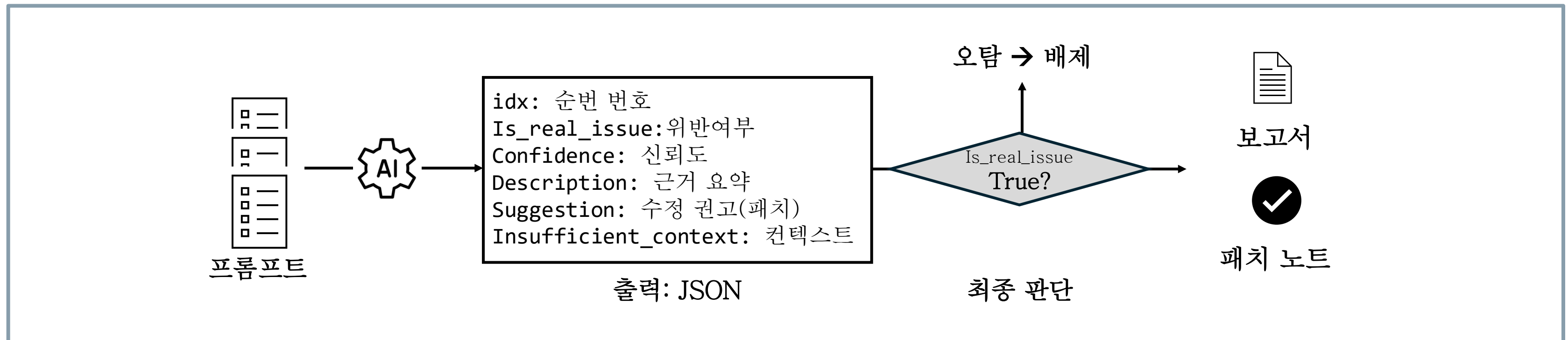
- 각 룰셋에 따른 가이드라인과 Few-shot을 구성하여 정확도를 높이하고자 함

계층	구조	목적
[1]	페르소나 선언 (암호모듈 전문 리뷰어)	LLM이 암호 보안 관점에서 판단
[2]	GCFS	코드베이스 맥락 보완
[3]	RAG 가이드라인	규칙의 요구사항 파악, 실제 근거
[4]	Few-shot (위반 예시-정상 예시-오탐 예시)	예시를 통한 경계 판단
[5]	판단 대상 코드	실제로 판단해야 할 코드 맥락
[6]	CoT 추론 지시	신중한 분석 유도
[7]	출력 형식 지정	JSON 형태 출력 지시

L3: LLM 기반의 의미론적 재판정

L1 위반 후보에 대해 L2 맥락 프롬프트를 기반으로 **오탐 여부를 최종 판정**

- L1은 결정론적이고 빠르지만, 코드 문맥 전체를 이해하는 데에는 한계가 존재
- L1의 필터를 통해 위반 후보를 뽑아내고, 전체 문맥을 보고 LLM이 L3에서 최종 결정



L3: LLM 최종 출력값

LLM의 최종 출력은 **JSON 구조로 위반 여부, 신뢰도, 판정 근거, 수정 권고 포함**

- **confidence(신뢰도)**와 **is_real_issue(위반여부)**를 중심으로 후속 검토 및 최종 판정

필드	설명	후속 처리
is_real_issue	위반 여부 판정 (true = 실제 위반 false = 오탐 (FP))	오탐의 경우 confidence + 2차 검증
confidence	판정 확신도 (0~100)	제거/유지 결정
description	판정 근거. 표준 조항을 근거	최종 위반의 근거로 포함
suggestion	수정 방향 제안	최종 위반 객체에 매핑
insufficient_context	코드 스니펫 부족 여부 (true = 부족, false = 부족하지 않음)	true 시 더 넓은 맥락으로 제공

L3: confidence 임계값 및 재판정

confidence는 LLM이 **자신의 판정에 대한 확신도**를 0~100으로 표현한 값

FP와 FN에 대해서, **FN 발생의 위험도**가 더 높음 → 오탐 제거에 대한 기준을 더 높임

- FP가 발생할 경우, 불필요한 수정을 진행하게 되는 시간 소모의 위험도
- FN이 발생할 경우, 실제 위반점이 누락되게 되는 치명적인 위험도

is_real_issue	confidence	처리	잘못 판단
true (위반 판정)	≥ 75	즉시 위반 확정	FP 발생 (과잉 탐지)
	< 75	재판정 (LLM 재호출)	
false (위반 X, 오탐 판정)	≥ 80	오탐 판정, 삭제	FN 발생 (위반 누락)
	< 80	재판정 (LLM 재호출)	

L3: 후속 검토 (이중 판정)

재판정의 경우, **추가 문맥을 포함한 프롬프트로 재판정을 수행함**

- 추가 문맥을 포함하여 LLM이 재판정하도록 함

```
{
  "is_real_issue": true,
  "confidence": 68,
  "description": "rand() 호출이 있으나, 테스트 코드인지 실제 암호 연산인지 불분명함."
}
```

- 답변에서 근거와 함께 최종 위반을 판단

```
{
  "is_real_issue": false,
  "confidence": 40,
  "description": "재검토 결과, 해당 rand() 호출은 테스트 벡터 생성에만 사용되며 실제 암호 연산(키/IV 생성)에는 사용되지 않음을 코드 전체 흐름에서 확인함.",
  "suggestion": "오탐"
}
```

추적성 검증

함수 선언·에러 상수와 설계서·시험서를 대조하여 **코드-문서의 일관성을 검증**

- KCMVP 인증의 경우 제출한 코드와 문서들의 일치 여부를 대조해보아야 함
- 정규식 패턴 매칭과 AST를 활용한 **코드 목록**을 설계하여 분석을 진행

불일치 유형	설명
허위 기재	설계서에는 존재하는데 코드에는 없는 함수 존재
누락	코드에는 존재하는데 설계서에는 없는 공개 API
구현 불일치	설계서의 오류 코드가 코드에 존재하지 않음
시험 부족	공개 함수의 절반 미만이 시험서에 언급

성능 지표

LEA 알고리즘과 보안 정책서를 기반으로 **의도적인 위반 사항을 주입**하여 평가

- C언어 구현체-설계서의 4세트, 사전 정의한 GT 183건을 기반으로 진행

성능 지표는 각 단계의 효과를 독립적으로 측정하여 **각 단계의 필요성을 증명**

규칙 기반 탐지 (L1) 성능

- True Positive (TP)
- False Negative (FN)
- 탐지율 (Recall)
- GT 외 탐지

LLM 재판정(L3) 성능

- L3 으로 인한 제거 건수
- L3 정확 제거율
- L3 오판 (FN유발)

실질적 보완 지원 성능

- 패치 성공률
- 문서 구조 누락 탐지

성능 평가

지표	결과
True Positive (TP)	150건
False Negative (FN)	33건
탐지율 (Recall)	82.0%
GT 외 추가 탐지 건	97건
L3 필터링 - 제거 건수	53건(36.6%)
L3 필터링 - 정확 제거율	94.3%(50/53)
L3 필터링 - 오판 (FN 유발)	3건(5.7%)
문서 구조 누락 탐지	평균 38.5건/세트
패치 성공률	89.5%(17/19)

테스트 결과 다음과 같은 경향을 보임

- **missing** 유형에서의 위반을 완전하게 탐지함
- L3는 **semantic** 계열에서 오탐을 효과적으로 줄였으나, **연산 순서를 검증하는** 과정에서 오탐이 발생
- **AST 전용 검사기**가 갖춰진 규칙은 복잡한 코드에서 오탐이 적었음

실제 화면 구성

암호 모듈 정적 분석 도구

GitHub 링크나 ZIP 파일을 업로드하면 KCMVP 룰로 소스를 분석하고, 위반 위치와 설명을 IDE 형태로 보여줍니다.

분석 설정 (암호 모듈 시험 신청서 기준)

구분	선택
보안수준	☒ 보안수준 1 ☐ 보안수준 2 ☐ 보안수준 3 ☐ 보안수준 4
제품구분	☐ 하드웨어 ☒ 소프트웨어 ☐ 펌웨어 ☐ 기타
암호알고리즘	☒ 블록암호 ☐ 해시 ☐ 메시지 인증코드 ☐ 난수발생기 ☐ 공개키 암호 ☐ 전자서명 ☐ 키 설정 ☐ 키 유도 ☐ 기타
암호 선택	☒ LEA ☐ ARIA ☐ AES
운영모드	☐ 선택 안 함 ☐ ECB ☒ CBC ☐ CTR ☐ OFB ☐ CFB ☐ CCM ☐ GCM ☐ CMAC

운영모드를 선택하면 해당 모드 룰만 적용됩니다. 선택 안 함 시 모드별 룰은 적용되지 않습니다.

GitHub 레포 분석

공개 레포지토리 URL을 붙여넣고 분석을 시작합니다. (현재는 ZIP 우선 지원)

https://github.com/user/repo

GitHub 링크로 분석

아래 한 줄에서 코드 ZIP과 3종 문서 PDF(설계서 / 형상관리 / 시험서)를 함께 업로드하세요.

코드 ZIP

설계서 PDF

형상관리 PDF

시험서 PDF

ZIP(코드) 1개 + PDF(문서) 3개까지 업로드할 수 있습니다. 문서는 텍스트 기반 PDF를 권장합니다.

ZIP 파일로 분석

03 성능평가 실제 화면 구성

KCMVP 사전 적합성 점검

+ 새 분석

ZIP: lea_cbc_only.zip파일 32개

위반 45건 (H 30 · M 14 · L 1)완료

보고서

설계서 14

형상관리

시험서

코드 28

src/arm_arch.h

src/lea_xop.h 3

src/lea.h 1

src/lea_cbc.h 4

src/lea_sse2.h

src/cpu_info.h

src/lea_avx2.h

src/lea_locl.h

src/lea_neon.h

lea_benchmark.c

benchmark.c

main_Otv.c

main.c

lea_vs.c

src/cpu_info_arm.c

src/lea_core.c 17

src/cpu_info_ia32.c

src/lea_t_avx2_sse2.c

src/lea_core_xop.c 1

src/lea_online.c 1

src/lea_t_xop.c

src/lea_t_sse2.c

src/lea_base.c

src/lea_t_neon.c

src/lea_t_fallback.c 1

선택된 파일: src/lea_core.c

전체 파일 AST 보기

파일 전체 흐름 보기

이 파일에는 파일 전체 수준의 위반(COM-001 등)이 있습니다. 잔존 정보 제거 코드가 전혀 없거나, 가이드라인에서 요구하는 처리 로직이 누락되어 있을 수 있습니다.

1#include "lea.h"

2#include "lea_locl.h"

3

4#include <stdio.h>

5

6static const unsigned int delta[8][36] = {

7{0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede, 0x3efe9dbc, 0x7dfd3b78, 0xfbfa76f0, 0xf7f4ede1,

패치 제안 LEA-025

문제 코드

```diff

static const unsigned int delta[8][36] = {

{0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede, 0x3efe9dbc, 0x7dfd3b78, 0xfbfa76f0, 0xf7f4ede1,

0xfe9dbc3, 0xdfd3b787, 0xbfa76f0f, 0x7f4ede1f, 0xfe9dbc3e, 0xfd3b787d, 0xfa76f0fb, 0xf4ede1f7,

0xe9dbc3ef, 0xd3b787df, 0xa76f0fbf, 0x4ede1f7f, 0x9dbc3efe, 0x3b787dfd, 0x76f0fbfa, 0xede1f7f4,

0xdc3efe9, 0xb787dfd3, 0x6f0fbfa7, 0xede1f7f4e, 0xbc3efe9d, 0x787dfd3b, 0xf0fbfa76, 0xe1f7f4eD,

0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede},

{0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812, 0x4626b024, 0x8c4d6048, 0x189ac091, 0x31358122,

0x626b0244, 0xc4d60488, 0x89ac0911, 0x13581223, 0x26b02446, 0xd60488c, 0x9ac09118, 0x35812231,

0x6b024462, 0xd60488c4, 0xac091189, 0x58122313, 0xb0244626, 0x60488c4d, 0xc091189a, 0x81223135,

0x0244626b, 0x0488c4d6, 0x091189ac, 0x12231358, 0x244626b0, 0x488c4d60, 0x91189ac0, 0x22313581,

0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812},

{0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453, 0x9e27c8a7, 0x3c4f914f, 0x789f229e, 0xf13e453c,

0xe27c8a79, 0xc4f914f3, 0x89f229e7, 0x13e453cf, 0x27c8a79e, 0x4f914f3c, 0x9f229e78, 0x3e453cf1,

0x7c8a79e2, 0xf914f3c4, 0xf229e789, 0xe453cf13, 0xc8a79e27, 0x914f3c4f, 0x229e789f, 0x453cf13e,

0x8a79e27c, 0x14f3c4f9, 0x29e789f2, 0x53cf13e4, 0xa79e27c8, 0x4f3c4f91, 0x9e789f22, 0x3cf13e45,

0x79e27c8a. 0xf3c4f914. 0xe789f229. 0xcf13e453}.

요약

위반 목록

패치

분석 완료

LEA-025 줄 7

LEA-025-src\_lea\_core.c-7.md

문제 코드

0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453}, {0x78df30ec, 0xf1be61d8, 0xe37cc3b1, 0xc6f98763, 0x8df30ec7, 0x1be61d8f, 0x37cc3b1e, 0x6f98763c, 0xdf30ec78, 0xbe61d8f1, 0x7cc3b1e3, 0xf98763c6, 0xf30ec78d, 0xe61d8f1b, 0xcc3b1e37, 0x98763c6f, 0x30ec78df, 0x61d8f1be, 0xc3b1e37c, 0x8763c6f9, 0xec78df3, 0xd8f1be6, 0x3b1e37cc, 0x763c6f98, 0xec78df30, 0xd8f1be61, 0xb1e37cc3,

수정 코드

0x78df30ec, + // 5[4] = 0x715ea49e, 5[5] = 0xc785da0a, 5[6] = 0xe04ef22a, 5[7] = 0xe5c40957 + // 유도 근거: 'L'(76), 'E'(69), 'A'(95)의 ASCII 코드와 √766995의 소수점 이하 값에서 도출됨. +static const unsigned int delta[8] = { + 0xc3efe9db, 0x44626b02, 0x79e27c8a, 0x78df30ec, + 0x715ea49e, 0xc785da0a, 0xe04ef22a, 0xe5c40957 +};

수정 이유

기존 코드는 LEA 키 스케줄에 사용되는 상수 5[0]~5[7]의 값이 KCMVP 규격에서 요구하는 표준 값과 다릅니다. KCMVP 규격 (LEA-011)에 따라 정확한 표준 상수 값으로 수정해야 하며, 상수 값의 유도 근거를 주석으로 명시하여 투명성을 확보해야 합니다.

코드에서 보기

LEA-025 줄 63

펼치기

AI 기반 KCMVP 사전 검증 도구: 규칙 탐지와 LLM 의미론적 분석의 하이브리드 모델

23



# 실제 화면 구성

KCMVP 사전 적합성 점검

ZIP: lea\_cbc\_only.zip

파일 32개

위반 45건 (H 30 · M 14 · L 1)

완료

보고서

설계서 14

형상관리

시험서

코드 28

src/arm\_arch.h

src/lea\_xop.h 3

src/lea.h 1

src/lea\_cbc.h 4

src/lea\_sse2.h

src/cpu\_info.h

src/lea\_avx2.h

src/lea\_locl.h

src/lea\_neon.h

lea\_benchmark.c

benchmark.c

main\_Otv.c

main.c

lea\_vs.c

src/cpu\_info\_arm.c

src/lea\_core.c 17

src/cpu\_info\_ia32.c

src/lea\_t\_avx2\_sse2.c

src/lea\_core\_xop.c 1

src/lea\_online.c 1

src/lea\_t\_xop.c

src/lea\_t\_sse2.c

src/lea\_base.c

src/lea\_t\_neon.c

src/lea\_t\_fallback.c 1

선택된 파일: src/lea\_core.c

전체 파일 AST 보기

파일 전체 흐름 보기

이 파일에는 파일 전체 수준의 위반(COM-001 등)이 있습니다. 잔존 정보 제거 코드가 전혀 없거나, 가이드라인에서 요구하는 처리 로직이 누락되어 있을 수 있습니다.

```

1 #include "lea.h"
2 #include "lea_locl.h"
3
4 #include <stdio.h>
5
6 static const unsigned int delta[8][36] = {
7 {0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede, 0x3efe9dbc, 0x7dfd3b78, 0xfbfa76f0, 0xf7f4ede1,
8 0xfe9dbc3, 0xdfd3b787, 0xbfa76f0f, 0x7f4ede1f, 0xfe9dbc3e, 0xfd3b787d, 0xfa76f0fb, 0xf4ede1f7,
9 0xe9dbc3ef, 0xd3b787df, 0xa76f0fbf, 0x4ede1f7f, 0x9dbc3efe, 0x3b787dfd, 0x76f0fbfa, 0xed1f7f4,
10 0xdc3efe9, 0xb787dfd3, 0x6f0fbfa7, 0xde1f7f4e, 0xbc3efe9d, 0x787dfd3b, 0xf0fbfa76, 0xe1f7f4eD,
11 0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede},
12 {0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812, 0x4626b024, 0x8c4d6048, 0x189ac091, 0x31358122,
13 0x626b0244, 0xc4d60488, 0x89ac0911, 0x13581223, 0x26b02446, 0xd60488c, 0x9ac09118, 0x35812231,
14 0x6b024462, 0xd60488c4, 0xac091189, 0x58122313, 0xb0244626, 0x60488c4d, 0xc091189a, 0x81223135,
15 0x0244626b, 0x0488c4d6, 0x091189ac, 0x12231358, 0x244626b0, 0x488c4d60, 0x91189ac0, 0x22313581,
16 0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812},
17 {0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453, 0x9e27c8a7, 0x3c4f914f, 0x789f229e, 0xf13e453c,
18 0xe27c8a79, 0xc4f914f3, 0x89f229e7, 0x13e453cf, 0x27c8a79e, 0x4f914f3c, 0x9f229e78, 0x3e453cf1,
19 0x7c8a79e2, 0xf914f3c4, 0xf229e789, 0xe453cf13, 0xc8a79e27, 0x914f3c4f, 0x229e789f, 0x453cf13e,
20 0x8a79e27c, 0x14f3c4f9, 0x29e789f2, 0x53cf13e4, 0xa79e27c8, 0x4f3c4f91, 0x9e789f22, 0x3cf13e45,
21 0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453},

```

패치 제안 LEA-025

### 문제 코드

```diff

static const unsigned int delta[8][36] = {

{0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede, 0x3efe9dbc, 0x7dfd3b78, 0xfbfa76f0, 0xf7f4ede1,

0xfe9dbc3, 0xdfd3b787, 0xbfa76f0f, 0x7f4ede1f, 0xfe9dbc3e, 0xfd3b787d, 0xfa76f0fb, 0xf4ede1f7,

0xe9dbc3ef, 0xd3b787df, 0xa76f0fbf, 0x4ede1f7f, 0x9dbc3efe, 0x3b787dfd, 0x76f0fbfa, 0xed1f7f4,

0xdc3efe9, 0xb787dfd3, 0x6f0fbfa7, 0xde1f7f4e, 0xbc3efe9d, 0x787dfd3b, 0xf0fbfa76, 0xe1f7f4eD,

0xc3efe9db, 0x87dfd3b7, 0x0fbfa76f, 0x1f7f4ede},

{0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812, 0x4626b024, 0x8c4d6048, 0x189ac091, 0x31358122,

0x626b0244, 0xc4d60488, 0x89ac0911, 0x13581223, 0x26b02446, 0xd60488c, 0x9ac09118, 0x35812231,

0x6b024462, 0xd60488c4, 0xac091189, 0x58122313, 0xb0244626, 0x60488c4d, 0xc091189a, 0x81223135,

0x0244626b, 0x0488c4d6, 0x091189ac, 0x12231358, 0x244626b0, 0x488c4d60, 0x91189ac0, 0x22313581,

0x44626b02, 0x88c4d604, 0x1189ac09, 0x23135812},

{0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453, 0x9e27c8a7, 0x3c4f914f, 0x789f229e, 0xf13e453c,

0xe27c8a79, 0xc4f914f3, 0x89f229e7, 0x13e453cf, 0x27c8a79e, 0x4f914f3c, 0x9f229e78, 0x3e453cf1,

0x7c8a79e2, 0xf914f3c4, 0xf229e789, 0xe453cf13, 0xc8a79e27, 0x914f3c4f, 0x229e789f, 0x453cf13e,

0x8a79e27c, 0x14f3c4f9, 0x29e789f2, 0x53cf13e4, 0xa79e27c8, 0x4f3c4f91, 0x9e789f22, 0x3cf13e45,

0x79e27c8a, 0xf3c4f914, 0xe789f229, 0xcf13e453},

요약

위반 목록

패치

분석 완료

전체 28

확정 12

후보 16

HIGH 확정 COM-001 lea.h:항목부재

잔존 정보 제거(Zeroization)

HIGH 확정 COM-001 lea_online.c:항목부재

잔존 정보 제거(Zeroization)

MED 확정 COM-002 lea_t_fallback.c:4

예러 처리 및 동일 예러 코드

MED 확정 COM-006 lea_xop.h:466

암호 함수 표준 명칭(lea_*) 사용

MED 확정 COM-006 lea_xop.h:537

암호 함수 표준 명칭(lea_*) 사용

MED 확정 COM-006 lea_xop.h:573

암호 함수 표준 명칭(lea_*) 사용

MED 후보 LEA-008 lea_core.c:항목부재

내부 상태 변수(Xi) 4개 워드 배열 구성

MED 후보 LEA-009 lea_core.c:항목부재

라운드키(RKi) 6개 워드(192비트) 구성

HIGH 후보 LEA-010 lea_core.c:전체

delta 상수 배열 'delta': KS X 3246 비표준 값 포함 — 비표준: [0]={, [1]={, [2]={, [3]={. 표준 8개: 0xc3efe9db, 0x44626b02, 0x79e27c8a, 0x78df30ec...

03 성능 평가

실제 화면 구성

KCMVP 사전 적합성 점검

+ 새 분석

ZIP: lea_cbc_only.zip파일 32개

위반 45건 (H 30 · M 14 · L 1) ● 완료

보고서

설계서 14

형상관리

시험서

코드 28

1.6 암호모듈 명칭 및 버전 정보 (...)

2.1 논리적 인터페이스 연관관계 ...

2.2 서비스별 논리적 인터페이스 ...

2.3 특정 상태 시 입출력 금지 보...

2.4 상태 출력 및 오류 코드 식별 ...

2.5 가변 길이 입력 및 데이터 형...

3.1 동시 운영자 지원 여부 (AS0...

3.2 역할 정의 (AS04.05, AS0...

3.3 서비스별 인가된 역할 매핑 (...)

3.4 버전 정보 출력 서비스 (AS...

3.5 상태 표시 및 확인 서비스 (A...

3.6 사용자 호출 방식 동작 전 자...

3.7 제로화 서비스 (AS04.17)

3.8 리셋/전원 종료 시 이전 인증...

3.9 인증 데이터 보호 (AS04.44)

3.10 운영체제 연계 역할 할당 (...)

4.1 소프트웨어 무결성 검증 기법...

4.2 무결성 검증키 분산 저장 (A...

4.3 무결성 실패 시 오류 천이 및 ...

4.4 온디맨드 무결성 시험 API (...)

4.5 안전한 배포 및 설치 전 무결...

5.1 운영환경 상세 정보 (AS06....

5.2 프로세스 및 가상 메모리 보...

5.3 자식 프로세스 생성 (AS06....

6.1 CSP 및 PSP 보호 (AS09....

6.2 난수발생기 상태 정보의 CS...

6.3 외부 엔트로피 잡음원의 CS...

6.4 SSP 생성 및 자동화된 SSP...

6.5 SSP 주입 및 출력 형태 (AS...

6.6 SSP 저장 및 PSP 변경 금...

6.7 사용 후 메모리 정리 (AS09...

7.1 자가시험 실패 시 오류 처리 ...

7.2 자가시험 오류 시 암호 서비...

설계서 뷰어

docs/design/fake_design_doc.pdf

마크다운 모드

● 위반 14건

● [DOC-003] 검증대상 암호알고리즘 및 핵심보안매개변수(CSP) 명세 — 필수 항목 누락

확정

● [DOC-012] 서비스별 논리적 인터페이스 상세 명세 — 필수 항목 누락

확정

● [DOC-022] 제로화 서비스 제공 및 동작 명세 — 필수 항목 누락

확정

▼ 나머지 11건 더 보기

본문

LEA 암호모듈 기본 및 상세설계서 문서 버전: V1.2 작성일: 2026년 3월 19일 대상 모듈: KISA-LEA 소프트웨어 암호모듈 보안수준: 1

암호모듈 명세

| 파일명 | 역할 |
|----------------|--------------------------|
| lea_core.c | LEA 블록암호 핵심 연산 (암호화/복호화) |
| lea_cbc.c | CBC 운영모드 구현 |
| lea_ctr.c | CTR 운영모드 구현 |
| lea_gcm.c | GCM 운영모드 구현 |
| lea_selftest.c | 동작 전 자가시험 및 KAT |
| lea_zeroize.c | 보안 메모리 정리 구현 |

요약

위반 목록

패치

분석 완료

SSP 생성 및 자동화된 SSP 설정(합의/전송) 방법 명세

MED

확정

DOC-039

설계서 · 누락

SSP 저장 및 PSP 보호(변경 금지) 명세

HIGH

확정

DOC-040

설계서 · 누락

절차적/운영적 제로화 및 복구 불가능성 명세

HIGH

확정

DOC-042

설계서 · 누락

자가시험 오류 시 암호 서비스 제한 및 출력 금지 명세

HIGH

확정

DOC-043

설계서 · 누락

소프트웨어 무결성 시험 전 무결성 기술 자가시험 선행 명세

HIGH

확정

DOC-046

설계서 · 누락

조건부 키쌍 일치 시험 수행 방법 명세

HIGH

확정

DOC-048

설계서 · 누락

유한상태모델(상태천이도 및 상태천이표) 정의

MED

확정

DOC-049

설계서 · 누락

단순 오류 발생 시 복구 및 자동 천이 절차 명세

HIGH

확정

DOC-051

설계서 · 누락

기능시험 및 자동화 보안 진단(취약점 점검) 명세

HIGH

확정

DOC-052

설계서 · 누락

수명 종료 시의 안전한 소거 절차 상세 서술

HIGH

확정

DOC-054

설계서 · 누락

공개키 타이밍 공격 및 키 노출(마스킹) 대응 명세

AI 기반 KCMVP 사전 검증 도구: 규칙 탐지와 LLM 의미론적 분석의 하이브리드 모델

25

03 성능 평가 실제 화면 구성

KCMVP 사전 적합성 점검

ZIP: lea_cbc_only.zip파일 32개

📄 보고서

설계서 14

형상관리

시험서

코드 28

KCMVP 사전 적합성 분석 보고서

대상: LEA · 모드: CBC · 2026-04-28

✖ 불합격

↓ PDF 다운로드

🖨 인쇄

추적성 (TRC)

0건

3건

⚠ 검토 필요

🤖 AI 종합 평가

종합 판정: 불합격 위험

핵심 위험 요소

- [COM-001] 잔존 정보 제거(Zeroization) 미흡은 민감 정보 유출 위험을 야기하여 보안에 치명적입니다.
- [LEA-010] delta 상수 배열의 비표준 값 포함은 암호 알고리즘의 예측 불가능성과 안전성을 심각하게 저해할 수 있습니다.
- [LEA-048] KAT/MMT/MCT REQUEST 파일 형식 규격 미준수는 검증 과정의 신뢰성을 훼손하고 재현성을 어렵게 합니다.

우선 수정 권고

- [COM-001] 잔존 정보 제거(Zeroization) 기능의 완전한 구현 및 검증을 최우선으로 해야 합니다.
- [LEA-010] delta 상수 배열의 표준 값 적용 및 [LEA-015] delta 인덱싱 패턴의 올바른 구현을 통해 알고리즘의 무결성을 확보해야 합니다.
- [LEA-048], [LEA-049], [LEA-050], [LEA-058], [LEA-060], [LEA-062] 등 KAT 관련 규격 준수 및 구현 오류를 수정하여 검증 프로세스의 신뢰성을 높여야 합니다.

전반적 평가

본 암호모듈은 다수의 심각한 위반 사항을 포함하고 있어 현재 상태로는 KCMVP 적합성이 매우 낮습니다. 특히 잔존 정보 제거 미흡, 비표준 상수 사용, 그리고 검증 파일 형식 규격 미준수는 보안 취약점 및 검증 신뢰성 저하로 직결될 수 있는 중대한 문제입니다. 제시된 핵심 위험 요소 및 우선 수정 권고 사항을 철저히 반영하여 개선 작업을 진행해야 KCMVP 인증을 기대할 수 있을 것입니다.

공통 보안 (COM) 6건

✖ 불합격

▼ lea_online.c src/lea_online.c

COM-001HIGH확정

항목 부재

잔존 정보 제거(Zeroization)

결론

본 연구에서는 **퍼널 구조의 AI 기반 KCMVP 사전 검증 시스템**을 구현

- KCMVP 사전 검증 공백 보완을 위한 **LLM 기반 3단계 자동 분석 파이프라인 설계**
- **AST·심볼 그래프 주입, LLM 이중 검증, RAG, 계층적 프롬프트 설계** 등을 이용하여 정확도를 향상함
- 실제 KCMVP 규격 문서를 기반으로 룰셋을 직접 정의하고, 반복 실험을 통해 오판 패턴을 규정화하여 확장 가능한 **구조적 룰셋 및 오탐 후보 기준을 체계화**
- 그러나 **실제 KCMVP 검증 제도 문서 및 암호모듈 소스코드 데이터셋을 확보하지 못하였다는 한계**가 존재하며, 이를 위한 데이터셋 확보 및 정밀성 향상 목표

참고문헌

- [1] 국가정보원, "암호모듈 시험 및 검증지침"
- [2] 대한민국 대통령령, "전자정부법 시행령 제69조," 2025
- [3] NIST, FIPS 140-2: Security Requirements for Cryptographic Modules, 2001
- [4] NIST, FIPS 140-3: Security Requirements for Cryptographic Modules, 2019
- [5] KS X ISO/IEC 19790:2015 — 암호모듈 보안 요구사항
- [6] KS X ISO/IEC 24759:2015 — 암호모듈 시험 요구사항
- [7] NSR, "암호모듈 사전검증 서비스 사용자 가이드," v1.0, 2023
- [8] D. Hong et al., "LEA: A 128-Bit Block Cipher," WISA 2013, LNCS 8267
- [9] NIST, FIPS 197: Advanced Encryption Standard (AES), 2023 updated
- [10] TTA, TTAS.KO-12.0004/R1: 128-bit Symmetric Block Cipher (SEED), 2005
- [11] W.X. Zhao et al., "A Survey of Large Language Models," arXiv:2303.18223, 2023
- [12] Google, "Expanding the Gemini 2.5 Family of Models," Google Blog, 2025
- [13] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020
- [14] S. Rahaman et al., "CryptoGuard: High Precision Detection of Crypto Vulnerabilities," ACM CCS 2019
- [15] S. Krüger et al., "CogniCrypt: Supporting Developers in Using Cryptography," IEEE/ACM ASE 2017
- [16] E. Bendersky, pycparser: Complete C99 parser in pure Python, github.com/eliben/pycparser



THANK YOU!

조수빈, 박도윤, 임다은, 김재환, 정수민, 형유림, 서화정